



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	M Raihan Al Lutfi
NPM	2428240156
Kelas	SI5C
Topik Tugas 1	Area Parking
Entitas utama	Artikel
Tautan repositori	raihannlutfi/si5c_kuis1_M.Raihan
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Area Parking
Entitas utama (tunggal)	Artikel
Alamat dasar	/articles
Field data	id, platNomor, jenisKendaraan, waktuMasuk, waktuKluar, biaya
Field wajib saat POST	platnomor,jeniskendaraan
Query string untuk filter	jeniskendaraan
Data awal	<pre>{ "id": 1, "platNomor": "BG 1234 AA", "jenisKendaraan": "motor", "waktuMasuk": "2026-09-24T08:15:00+07:00", "waktuKeluar": null, "biaya": 0 }, { "id": 2, "platNomor": "BG 5678 BB", "jenisKendaraan": "mobil", "waktuMasuk": "2026-09-24T09:00:00+07:00", "waktuKeluar": null, "biaya": 0 }</pre>
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[parking-records]	Tidak	200	Ambil semua data (dengan filter bila ada)

No	Metode	Alamat	API key	Status sukses	Fungsi
2	GET	/[parking-records]/:id	Tidak	200	Ambil satu data
3	POST	/[parking-records]	Ya	201	Tambah data
4	PUT	/[parking-records]/:id	Ya	200	Ubah data
5	DELETE	/[parking-records]/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah node -v, npm -v, npm init -y, dan npm install express cors dotenv beserta keluarannya sampai selesai.

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\kuis1 paw\si5c_kuis1_M.Raihan>node -v
v24.21.0

C:\kuis1 paw\si5c_kuis1_M.Raihan>npm -v
11.19.0

C:\kuis1 paw\si5c_kuis1_M.Raihan>npm init -y
Wrote to C:\kuis1 paw\si5c_kuis1_M.Raihan\package.json:

{
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^18.0.5",
    "express": "^5.2.1"
  },
  "name": "si5c_kuis1_m.raihan",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/raihannlutfi/si5c_kuis1_M.Raihan.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/raihannlutfi/si5c_kuis1_M.Raihan/issues"
  },
  "homepage": "https://github.com/raihannlutfi/si5c_kuis1_M.Raihan#readme",
  "devDependencies": {}
}

C:\kuis1 paw\si5c_kuis1_M.Raihan>npm install express

up to date, audited 72 packages in 3s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\kuis1 paw\si5c_kuis1_M.Raihan>npm install cors dotenv

up to date, audited 72 packages in 3s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\kuis1 paw\si5c_kuis1_M.Raihan>
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Express digunakan untuk membuat dan mengatur server serta REST API pada Node.js. Dengan Express, kita dapat membuat berbagai endpoint seperti GET, POST, PUT, dan DELETE dengan lebih mudah.

CORS digunakan untuk mengatur izin akses antara server dengan sumber atau domain yang berbeda. Misalnya, ketika aplikasi frontend dan API berada pada alamat yang berbeda, CORS membantu agar keduanya tetap dapat berkomunikasi.

Dotenv digunakan untuk membaca konfigurasi yang disimpan dalam file .env, seperti port server dan API key. Dengan begitu, informasi konfigurasi tidak perlu ditulis langsung di dalam kode program dan lebih mudah untuk diatur.

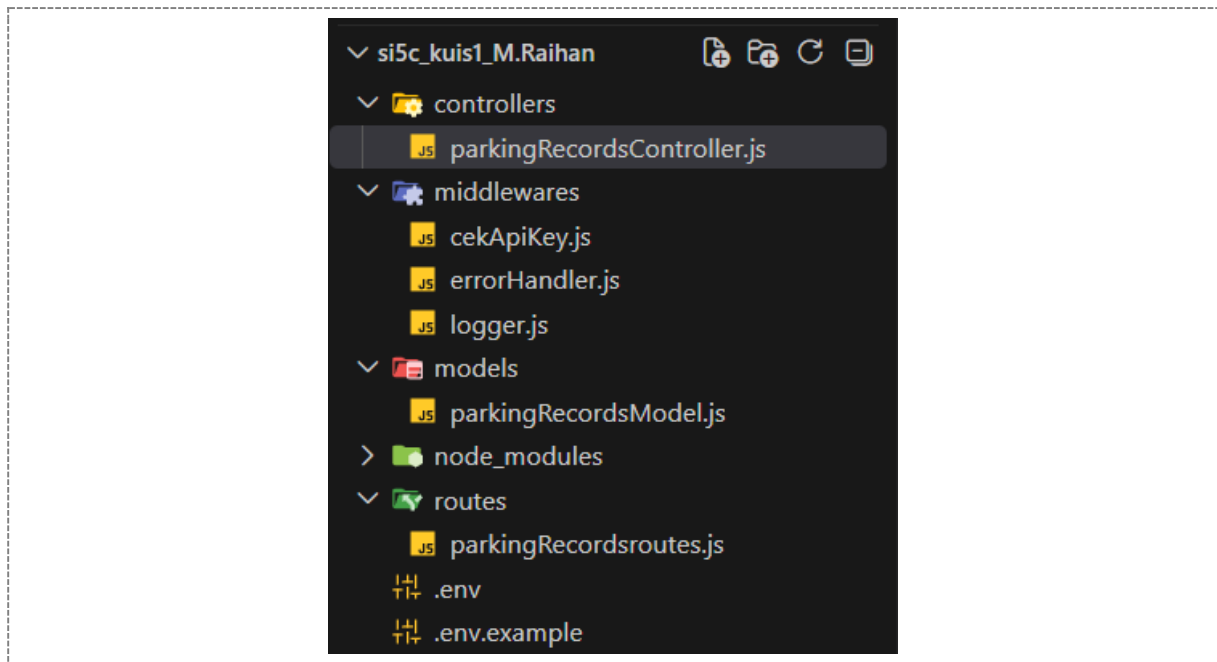
Versi Node.js perlu memenuhi syarat Express 5 karena Express 5 membutuhkan Node.js versi 18 atau lebih baru agar dapat berjalan dengan baik. Pada proyek ini digunakan Node.js versi **24.21.0**, sehingga sudah memenuhi persyaratan untuk menggunakan Express 5.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Folder **controllers** digunakan untuk mengatur proses ketika pengguna mengirim request ke API, seperti mengambil, menambah, mengubah, atau menghapus data area parkir, kemudian memberikan response kepada pengguna.

Folder **middlewares** digunakan sebagai perantara sebelum request diproses oleh controller, misalnya untuk mencatat aktivitas menggunakan logger, memeriksa API Key melalui cekApiKey, dan menangani kesalahan menggunakan errorHandler.

Folder **models** digunakan untuk menyimpan data area parkir dan menyediakan fungsi untuk mengolah data tersebut, seperti mengambil, menambah, mengubah, dan menghapus data tanpa berhubungan langsung dengan request dan response.

Folder **routes** digunakan untuk menentukan alamat atau endpoint API dan menghubungkan setiap endpoint dengan fungsi yang ada di controller, misalnya menentukan bahwa GET /parking-records digunakan untuk mengambil seluruh data parkir.

2.2 Ringkasan tugas setiap berkas

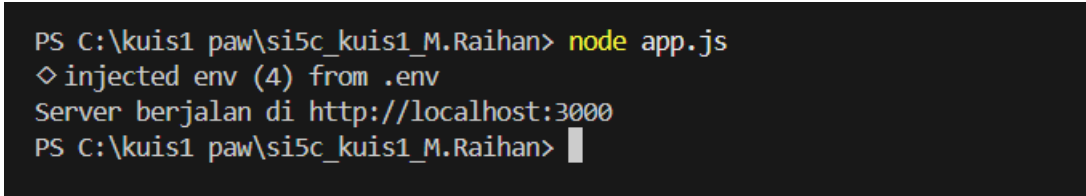
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/ parkingRecordsModel.js	Berfungsi sebagai tempat penyimpanan dan pengelolaan sementara data parking records dalam array. Berkas ini menyediakan fungsi seperti getAll, getByld, create, update, dan remove untuk melakukan operasi CRUD terhadap data parkir.
controllers/parkingRecordsControllers.js	Berfungsi menangani request dari pengguna dan memberikan response sesuai hasil proses. Berkas ini menggunakan fungsi dari parkingRecordsModel untuk mengambil, menambah, mengubah, dan menghapus data parking records serta melakukan validasi data sebelum diproses.
routes/parkingRecordsroutes.js	Berfungsi menentukan endpoint dan metode HTTP yang digunakan untuk mengakses data parking records. Berkas ini menghubungkan endpoint seperti GET, POST, PUT, dan DELETE dengan fungsi pada parkingRecordsController, serta menggunakan middleware cekApiKey pada operasi yang membutuhkan keamanan.
middlewares/logger.js	Berfungsi mencatat setiap request HTTP yang masuk ke server dengan menampilkan waktu, metode HTTP, dan URL yang diakses. Setelah mencatat request, fungsi logger menjalankan next() agar request dapat dilanjutkan ke proses berikutnya.
middlewares/cekApiKey.js	<i>Berfungsi memeriksa keamanan request dengan mengambil nilai x-api-key dari header dan membandingkannya dengan process.env.API_KEY. Jika API Key benar, fungsi next() dijalankan, sedangkan jika salah atau tidak ada, server memberikan response dengan status.</i>
middlewares/errorHandler.js	Berfungsi menangani error secara terpusat agar kesalahan pada aplikasi dapat memberikan response yang teratur. Berkas ini menyediakan notFoundHandler untuk menangani route yang tidak ditemukan dan errorHandler untuk menangani kesalahan seperti JSON yang tidak valid atau error pada server.
app.js	Berfungsi sebagai file utama yang menjalankan aplikasi Express dan mengatur seluruh konfigurasi server. Berkas ini memuat .env, memasang middleware seperti logger, cors, dan express.json(), menghubungkan parkingRecordsRoutes, serta memasang notFoundHandler dan errorHandler sebelum menjalankan server pada PORT yang ditentukan.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



```
PS C:\kuis1 paw\si5c_kuis1_M.Raihan> node app.js
◇ injected env (4) from .env
Server berjalan di http://localhost:3000
PS C:\kuis1 paw\si5c_kuis1_M.Raihan> █
```

Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port berasal dari konfigurasi PORT pada file `.env`, yaitu port 3000, sehingga server dapat diakses melalui `http://localhost:3000`. Jika port tersebut sudah digunakan oleh aplikasi lain, server tidak dapat berjalan dan akan menampilkan error karena port sedang digunakan.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

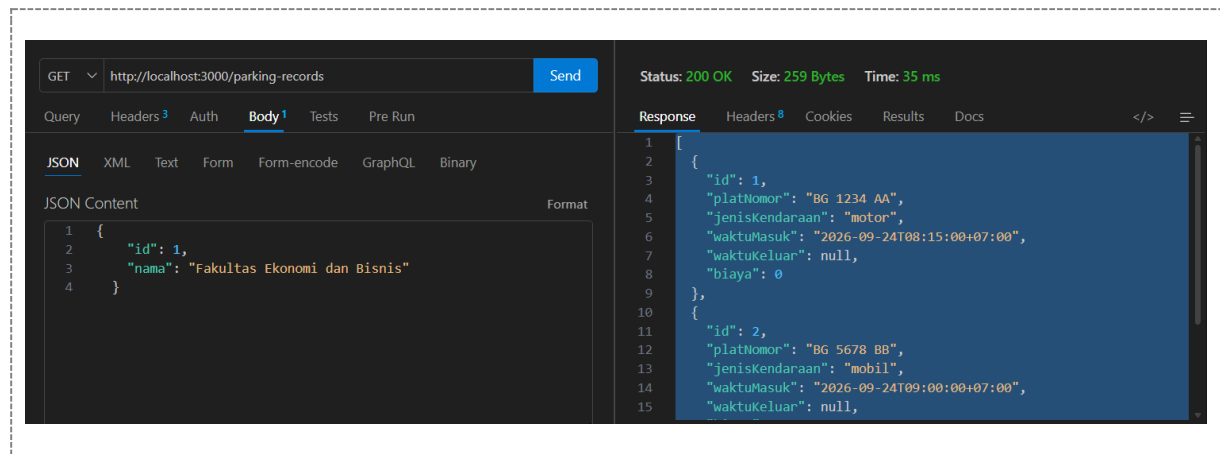
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /parking-records)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /parking-records
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	<pre> 200 OK[{ "id": 1, "platNomor": "BG 1234 AA", "jenisKendaraan": "motor", "waktuMasuk": "2026-09-24T08:15:00+07:00", "waktuKeluar": null, "biaya": 0 }, { "id": 2, "platNomor": "BG 5678 BB", "jenisKendaraan": "mobil", "waktuMasuk": "2026-09-24T09:00:00+07:00", "waktuKeluar": null, "biaya": 0 }]</pre>
Lulus?	☒ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

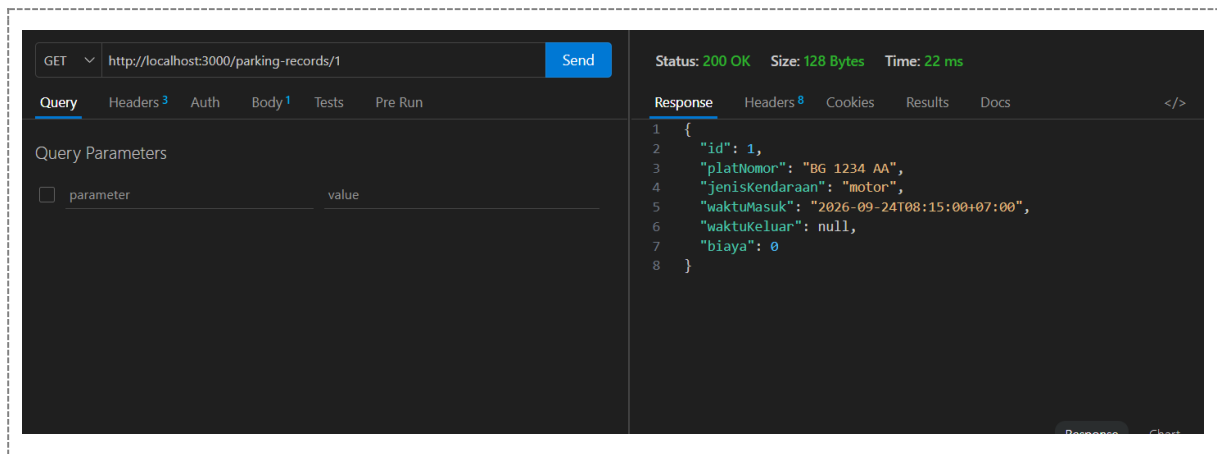
Penjelasan:

200 OK berarti request berhasil diproses oleh server. Endpoint berhasil mengambil dan menampilkan data parking records yang tersimpan. Pada foto terlihat beberapa data seperti nomor plat, jenis kendaraan, waktu masuk, waktu keluar, dan biaya.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /parking-records/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK { "id": 1, "platNomor": "BG 1234 AA", "jenisKendaraan": "motor", "waktuMasuk": "2026-09-24T08:15:00+07:00", "waktuKeluar": null, "biaya": 0 }
Lulus?	☒ Ya ☐ Tidak



Gambar 5. Hasil GET berdasarkan ID

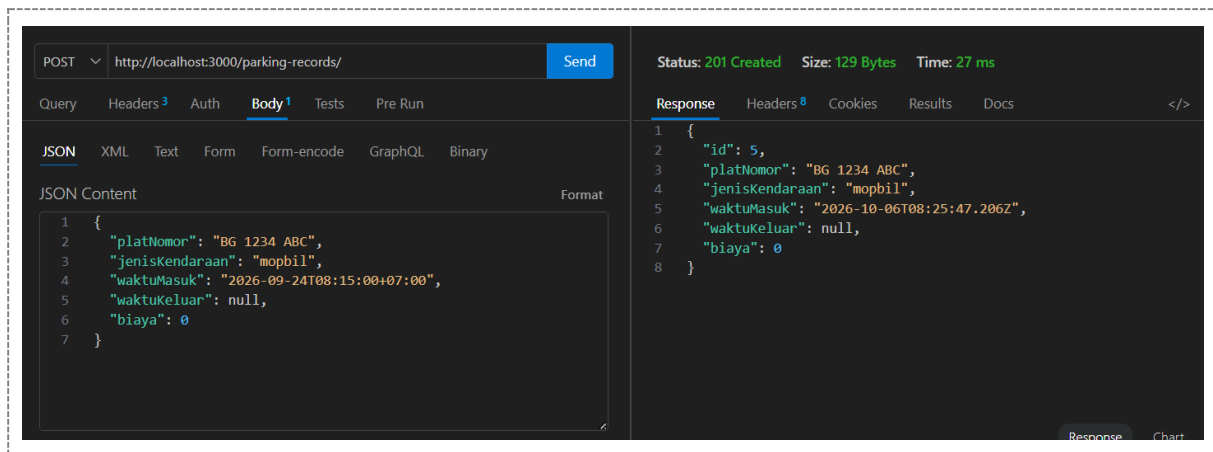
Penjelasan:

Pada endpoint GET /parking-records/1, angka 1 diambil dari alamat menggunakan req.params.id. Nilai dari req.params secara default berbentuk string, sehingga perlu diubah menjadi angka menggunakan parseInt(). Hal ini dilakukan karena ID pada data parking records disimpan dalam bentuk number. Setelah diubah menjadi angka, ID tersebut digunakan oleh Controller untuk memanggil fungsi getById() pada Model dan mencari data parking record dengan ID yang sesuai.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /parking-records/
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "platNomor": "BG 1234 ABC", "jenisKendaraan": "mopbil", "waktuMasuk": "2026-09-24T08:15:00+07:00", "waktuKeluar": null, "biaya": 0 }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created; { "id": 5, "platNomor": "BG 1234 ABC", "jenisKendaraan": "mopbil", "waktuMasuk": "2026-10-06T08:25:47.206Z", "waktuKeluar": null, "biaya": 0 }
Lulus?	☒ Ya ☐ Tidak



Gambar 6. Hasil POST data baru

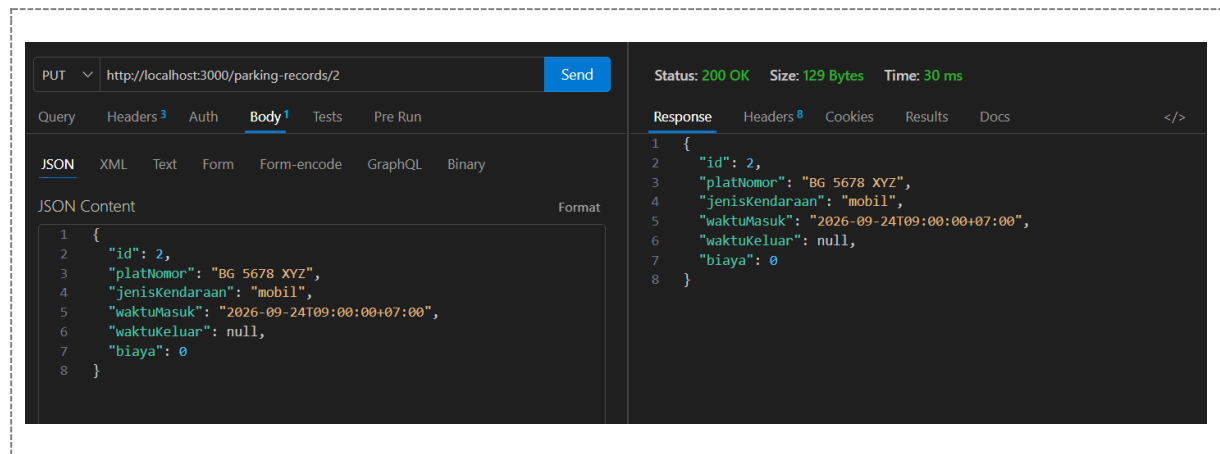
Penjelasan:

Header x-api-key berfungsi sebagai keamanan untuk endpoint POST, PUT, dan DELETE, yang diperiksa oleh middleware cekApiKey. Middleware tersebut memastikan API Key yang dikirim sesuai dengan API Key pada konfigurasi sebelum request diteruskan ke Controller. express.json() berfungsi agar Express dapat membaca data JSON yang dikirim melalui Body request. Pada pengujian POST, status yang digunakan adalah **201 Created** karena request berhasil membuat data parking record baru, sedangkan status 200 lebih umum digunakan untuk request yang berhasil diproses seperti mengambil data.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (<i>request</i>)	PUT /parking-records/2
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "id": 2, "platNomor": "BG 5678 XYZ", "jenisKendaraan": "mobil", "waktuMasuk": "2026-09-24T09:00:00+07:00", "waktuKeluar": null, "biaya": 0 }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK; { "id": 2, "platNomor": "BG 5678 XYZ", "jenisKendaraan": "mobil", "waktuMasuk": "2026-09-24T09:00:00+07:00", "waktuKeluar": null, "biaya": 0 }
Lulus?	☒ Ya ☐ Tidak



Gambar 7. Hasil PUT mengubah data

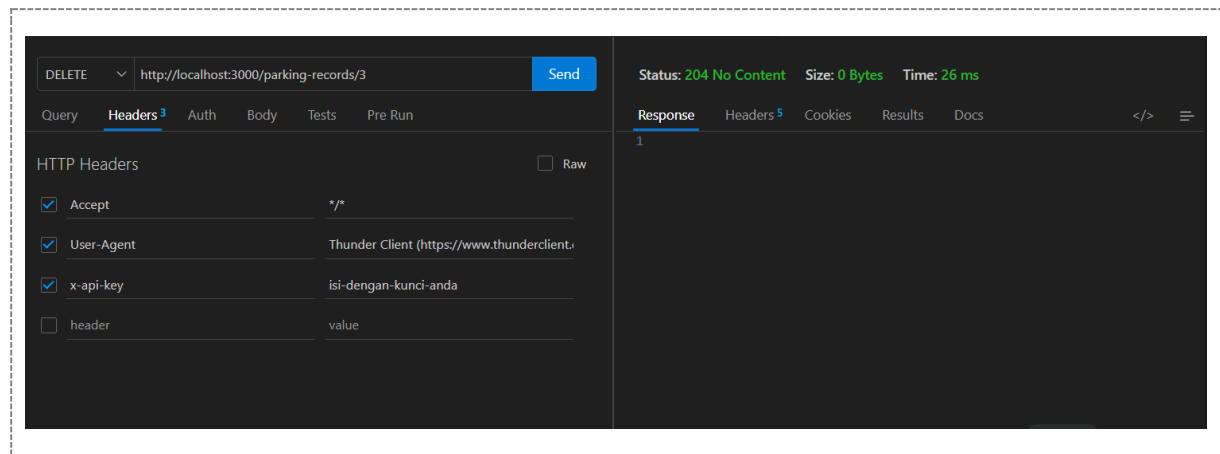
Penjelasan:

Pada pengujian PUT /parking-records/2, field yang mengalami perubahan adalah platNomor, dari BG 5678 BB menjadi BG 5678 XYZ. Sementara itu, field id, jenisKendaraan, waktuMasuk, waktuKeluar, dan biaya tetap. Proses perubahan melewati Route dan middleware cekApiKey, kemudian diteruskan ke fungsi update() pada Controller. Perubahan data sebenarnya dilakukan oleh fungsi update() pada Model, kemudian hasil perubahan dikembalikan oleh Controller dengan status **200 OK**.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /parking-records/3
Header	x-api-key: kunci-rahasia-paw2
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content
Lulus?	[✓] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Status **204 No Content** menunjukkan bahwa proses DELETE berhasil dilakukan, tetapi server tidak mengirimkan isi response. Pada pengujian ini, data parking record dengan ID 3 berhasil dihapus. Response tidak berisi data karena operasi DELETE hanya membutuhkan informasi bahwa proses penghapusan berhasil, sehingga server cukup mengembalikan status 204 tanpa mengirimkan JSON atau data tambahan.

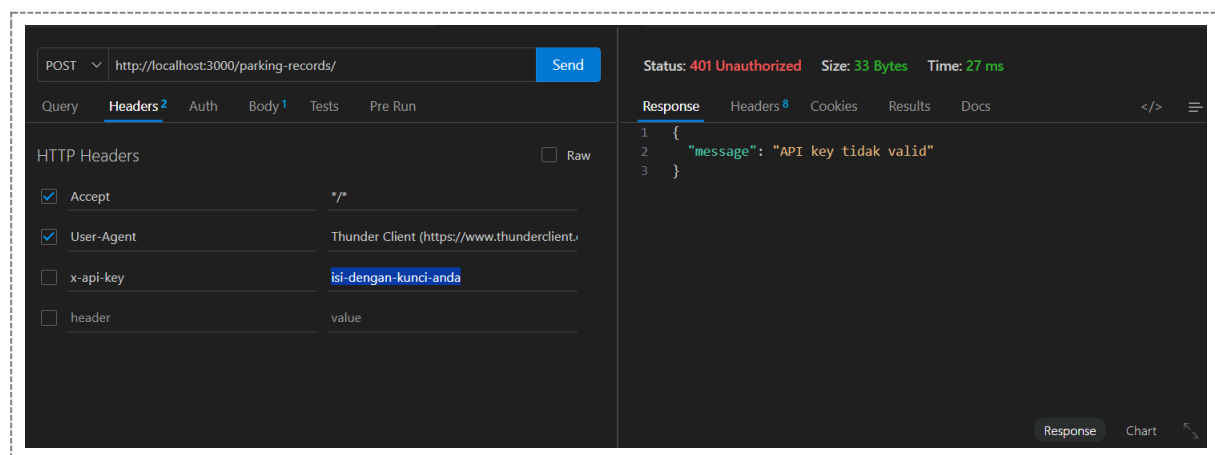
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /parking-records (tanpa x-api-key)
Header	Content-Type: application/json
Body	<pre>{ "platNomor": "BG 1234 ABC", "jenisKendaraan": "mopbil", "waktuMasuk": "2026-09-24T08:15:00+07:00", "waktuKeluar": null, "biaya": 0 }</pre>
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized; { "message": "API key tidak valid" }
Lulus?	[✓] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

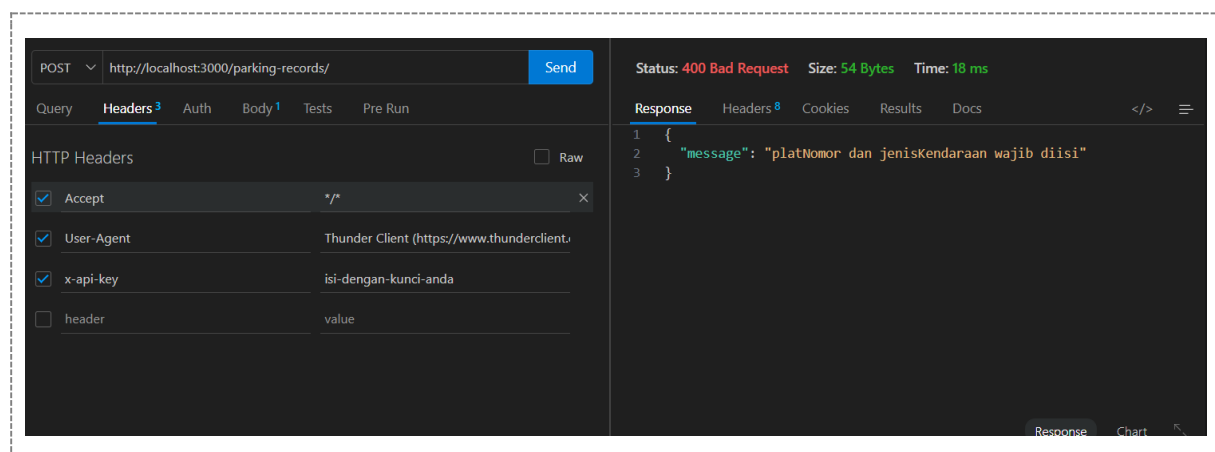
Penjelasan:

Pada pengujian POST `/parking-records/`, permintaan ditolak oleh middleware `cekApiKey` karena API Key yang dikirim tidak valid. Middleware memberikan response **401 Unauthorized** dengan pesan “API key tidak valid”. Controller tidak sempat dijalankan karena middleware berada sebelum Controller dan tidak meneruskan request ketika API Key tidak sesuai. Akibatnya, proses tidak dilanjutkan ke Model dan tidak terjadi perubahan atau penambahan data.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /parking-records/
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "platNomor": "BG 1234 ABC", "waktuMasuk": "2026-09-24T08:15:00+07:00", "waktuKeluar": null, "biaya": 0 }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request; { "message": "Semua field wajib diisi" }
Lulus?	[✓] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

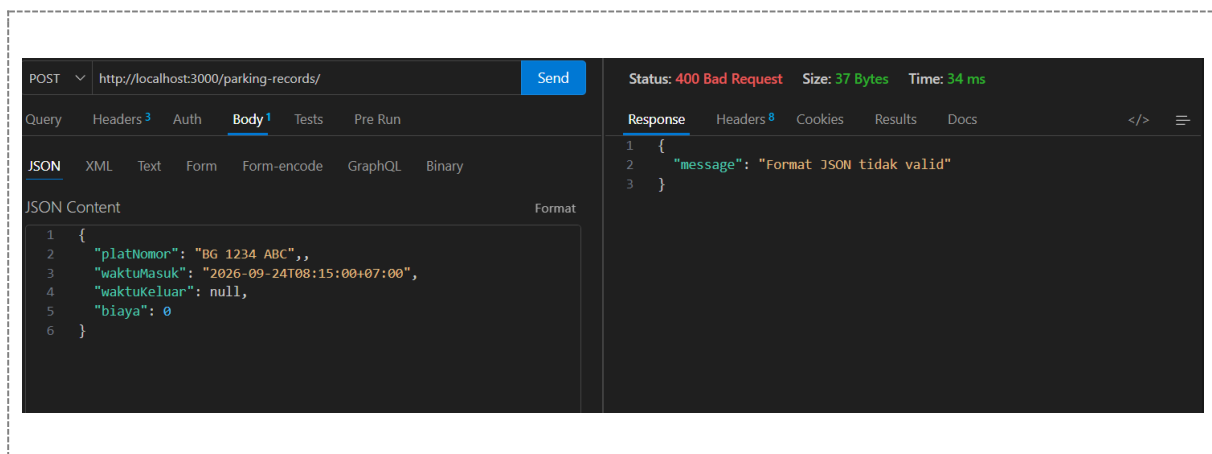
Penjelasan:

Validasi data `platNomor` dan `jenisKendaraan` ditulis pada layer **Controller**, yaitu pada fungsi `create()`. Controller memeriksa kelengkapan data sebelum proses dilanjutkan ke Model. Pada pengujian ini terdapat data wajib yang belum diisi sehingga Controller mengembalikan status **400 Bad Request** dengan pesan "platNomor dan jenisKendaraan wajib diisi". Status 400 digunakan karena kesalahan terdapat pada data request yang dikirim, bukan karena terjadi kesalahan pada server.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /articles
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "platNomor": "BG 1234 ABC",, "waktuMasuk": "2026-09-24T08:15:00+07:00", "waktuKeluar": null, "biaya": 0 }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request; { "message": "Format JSON tidak valid" }
Lulus?	☒ Ya ☐ Tidak



Gambar 11. POST dengan JSON rusak

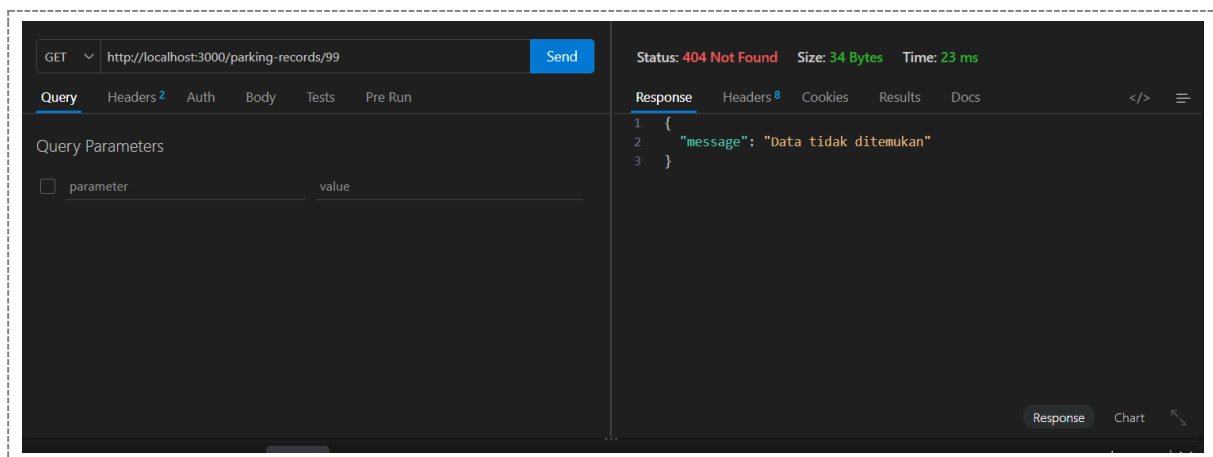
Penjelasan:

JSON yang tidak valid dideteksi oleh middleware `express.json()` ketika proses parsing Body request. Karena format JSON tidak dapat dibaca, error diteruskan ke errorHandler. Middleware errorHandler kemudian menangkap error tersebut dan mengubahnya menjadi response yang lebih rapi dengan status **400 Bad Request** dan pesan "Format JSON tidak valid". Karena error terjadi sebelum request mencapai Controller, fungsi Controller dan Model tidak dijalankan.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /parking-records/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found; { "message": "Data tidak ditemukan" }
Lulus?	[✓] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Pada pengujian GET /parking-records/99, request diterima oleh route GET /:id dan diteruskan ke fungsi getByld() pada Controller. Controller mengambil ID 99 dari parameter URL dan meminta Model mencari data tersebut. Karena Model tidak menemukan parking record dengan ID 99, hasil pencarian dikembalikan sebagai data yang tidak ditemukan. Controller kemudian mengirimkan response **404 Not Found** dengan pesan “Data tidak ditemukan”. Jadi, pada kasus ini response 404 dibuat dan dikirim oleh **Controller**, bukan oleh errorHandler.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	Ya
T02	Menampilkan satu data berdasarkan ID	200	200	Ya
T03	Menambah data baru	201	201	Ya
T04	Mengubah data	200	200	Ya
T05	Menghapus data	204	204	Ya
N01	Menambah data tanpa API key	401	401	Ya
N02	Menambah data dengan body tidak lengkap	400	400	Ya
N03	Mengirim JSON yang rusak	400	400	Ya
N04	Mengambil ID yang tidak ada	404	404	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/raihannlutfi/si5c_kuis1_M.Raihan
Pesan commit terakhir	Commit refactor
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

- **Route** : Route berfungsi sebagai pintu masuk request dengan menentukan endpoint dan mengarahkan request ke fungsi Controller yang sesuai, contohnya `router.get('/:id', parkingRecordsController.getById);`.
- **Controller** : Controller berfungsi mengatur proses request dan response, seperti mengambil ID, melakukan validasi, dan memanggil Model, contohnya `const data = parkingRecordsModel.getById(id);`
- **Model** : Model berfungsi mengelola data parking records, seperti mencari, menambah, mengubah, dan menghapus data, contohnya `const getById = (id) => parkingRecords.find(item => item.id === id);`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Penggunaan **error handler terpusat** membuat penanganan kesalahan lebih rapi dan tidak perlu menulis kode error yang sama di setiap route. Hal ini terbukti saat pengujian JSON tidak valid, sistem otomatis memberikan **400 Bad Request** dengan pesan "Format JSON tidak valid" melalui errorHandler. Dengan demikian, error dapat ditangani secara terpusat dan konsisten.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100